

Tarea Práctica — Clase 6

Tests y documentación dbt

Autor: @abarchello | Profesor: @rparrapy

MIAAD FPUNA 2026

1. Repositorio Git con tests y documentación

El proyecto dbt actualizado con tests y documentación completa está en:

- <https://github.com/abarchello/fpuna-datos-engineering>

2. dbt-expectations instalado

El paquete metaplane/dbt_expectations (versión 0.10.10) está declarado en dbt/packages.yml y se instala con dbt deps. También está godatadriven/dbt_date.

3. Tests genéricos implementados (más de 5)

Distribuidos en _stg__models.yml y _marts__models.yml:

- unique en PKs: pokemon_id, website_session_id, order_id, order_item_id, fecha_dia, etc.
- not_null en columnas obligatorias: estacion, fecha_medicion, utm_source, etc.
- accepted_values: device_type ∈ {mobile, desktop}, primary_type Pokémon, condiciones de calidad del agua.

En total se contabilizan más de 20 tests genéricos en el proyecto.

4. Tests dbt-expectations (más de 3)

- expect_column_values_to_be_between sobre rangos físicos en hidrometría (nivel 0-30 m, pH 0-14, turbidez 0-3000 NTU, OD 0-15 mg/L, temperatura 0-50 °C).
- expect_column_values_to_be_between sobre price_usd, margin_pct, conversion_rate_pct, total_base_stats, height_meters, weight_kg.
- expect_table_row_count_to_be_between sobre stg_hidro_mediciones.

Se contabilizan más de 10 tests de dbt-expectations.

5. Singular tests personalizados (3 implementados)

- tests/test_pokemon_no_duplicate_names.sql — verifica que no haya pokémons con el mismo nombre duplicado.
- tests/test_hidro_no_future_measurements.sql — verifica que ninguna medición tenga timestamp futuro (señal de error de reloj del sensor).
- tests/test_orders_margin_not_negative.sql — regla de negocio: ninguna orden debe tener margen negativo.

6. Documentación de modelos y columnas clave

Cada modelo está documentado en sus respectivos archivos YAML con description tanto a nivel modelo como a nivel columna. La documentación incluye explicación de las reglas de limpieza aplicadas en

staging (rangos físicos plausibles), motivación de cada mart, y semántica de las clasificaciones generadas (condicion_nivel, condicion_ph, power_tier).

7. Resultado de la ejecución completa de dbt build

El comando dbt build (que ejecuta deps + run + test) sobre TODOS los modelos del proyecto produce el siguiente resultado:

```
Done. PASS=83 WARN=8 ERROR=0 SKIP=0 NO-OP=0 TOTAL=91
```

Los 8 warnings están documentados y corresponden a outliers reales de los sensores hidrométricos (descalibración esperada) y a duplicados que aparecen por el modo Append del conector PokeAPI de Airbyte cuando se sincroniza varias veces el mismo Pokémon. Cero errores de ejecución.

8. Comandos de ejecución

El comando completo para reproducir el build con tests es:

```
$env:MOTHERDUCK_TOKEN = (Get-Content ..\.env | Where-Object {$_ -match
"^MOTHERDUCK_TOKEN"}).Split("=", 2)[1]

dbt deps --profiles-dir .
dbt build --profiles-dir .
```

La documentación interactiva se genera y sirve con:

```
dbt docs generate --profiles-dir .
dbt docs serve --profiles-dir .
```

Anexo — DAG generado por dbt docs serve

Captura del DAG (Lineage Graph) servido por dbt docs en localhost:8080. Muestra todas las fuentes (sources) en verde, modelos staging, intermediate y marts en celeste, y tests singulares en color distinto. Todas las dependencias entre modelos se trazan con líneas.



Figura — DAG completo del proyecto fpuna_datos_barchello